Unit-2
Unimodal and Multimodal Representations

MULTIMODAL MACHINE LEARNING

Language Representation

NLP

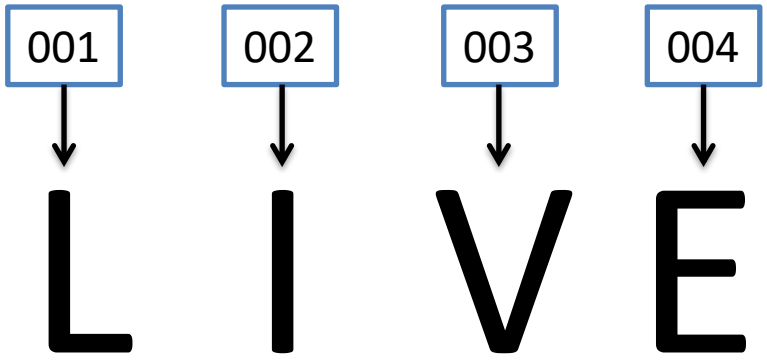
“You shall know a word by the company it keeps"
(J.R. Firth, 1957).

The meaning of a word is not an abstract "thing" in the head; it is a pattern of usage.

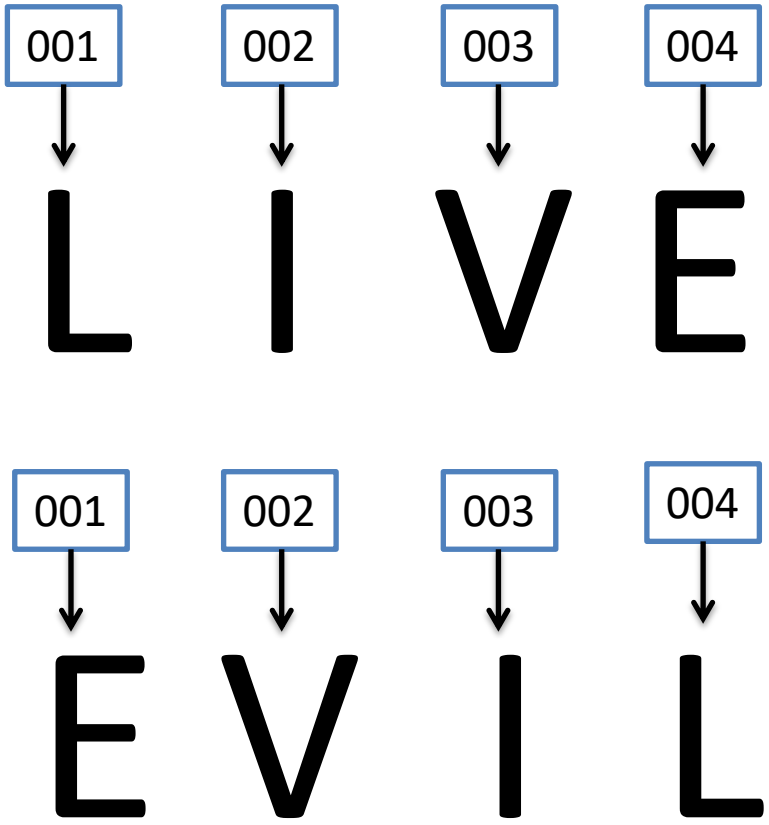
Language representation

- "He drank a glass of **bingle**."
- "The **bingle** was cold and refreshing."
- "We poured some **bingle** into the cup.“
- “Its **bingle** time”

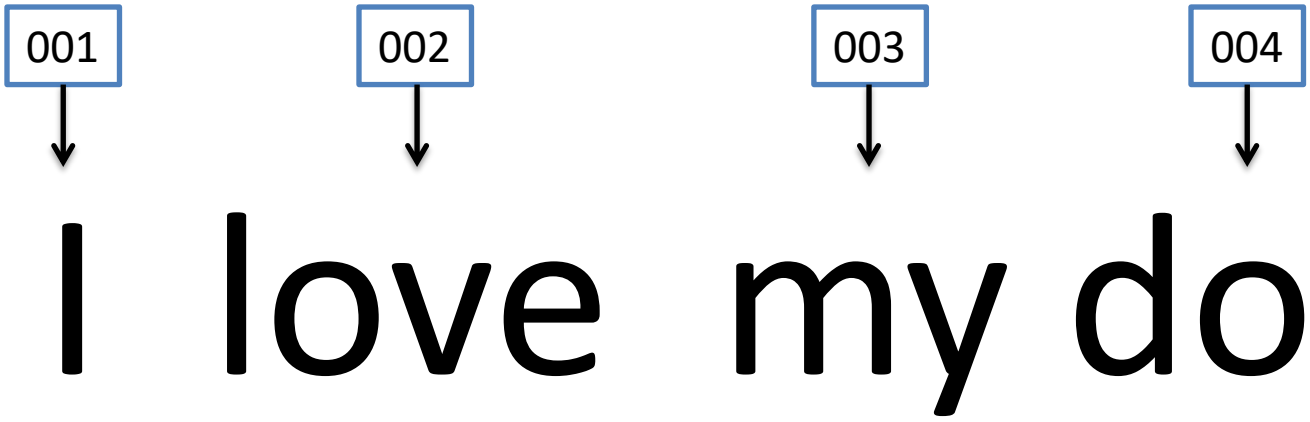
Word Embedding



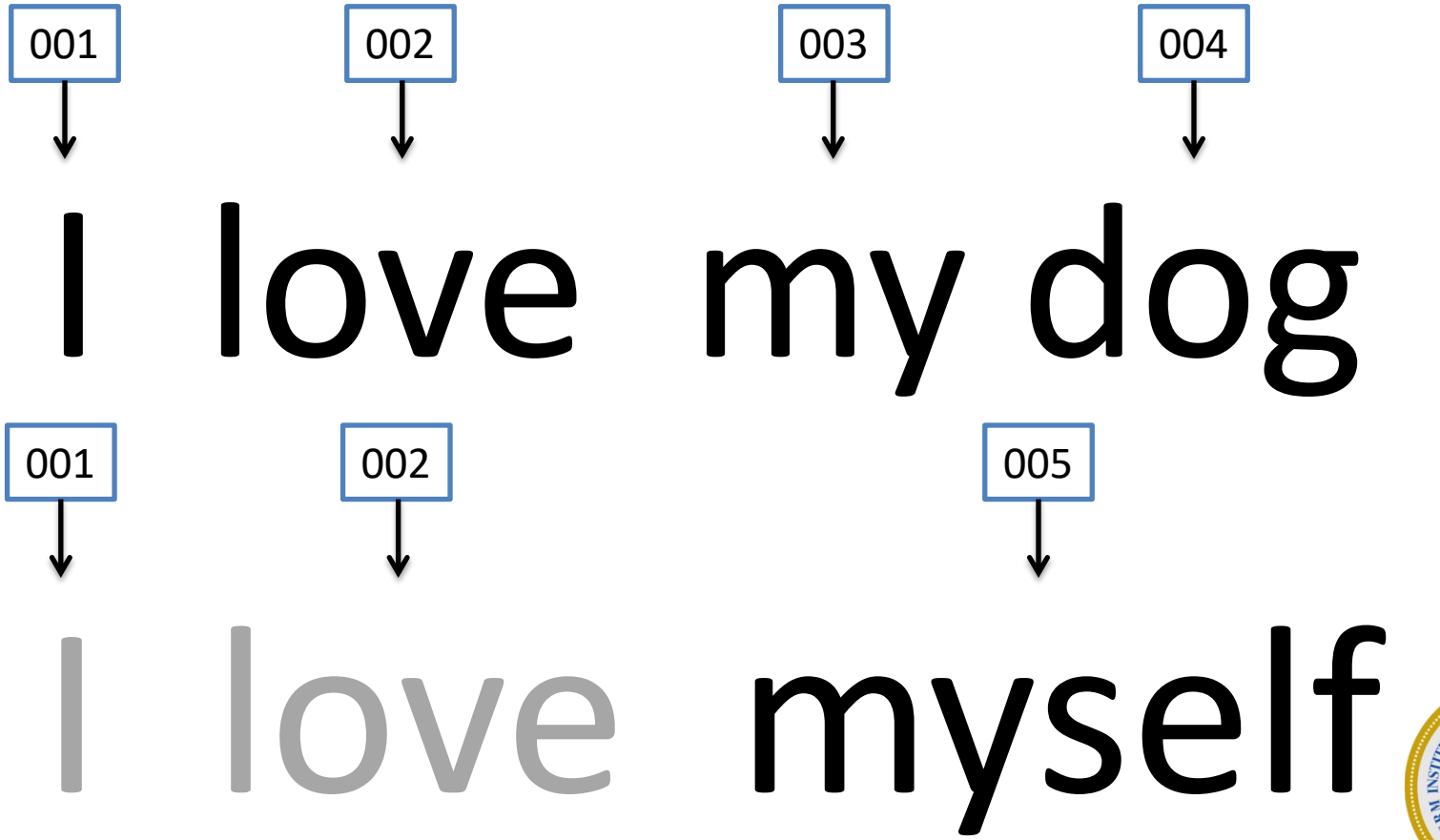
Word Embedding



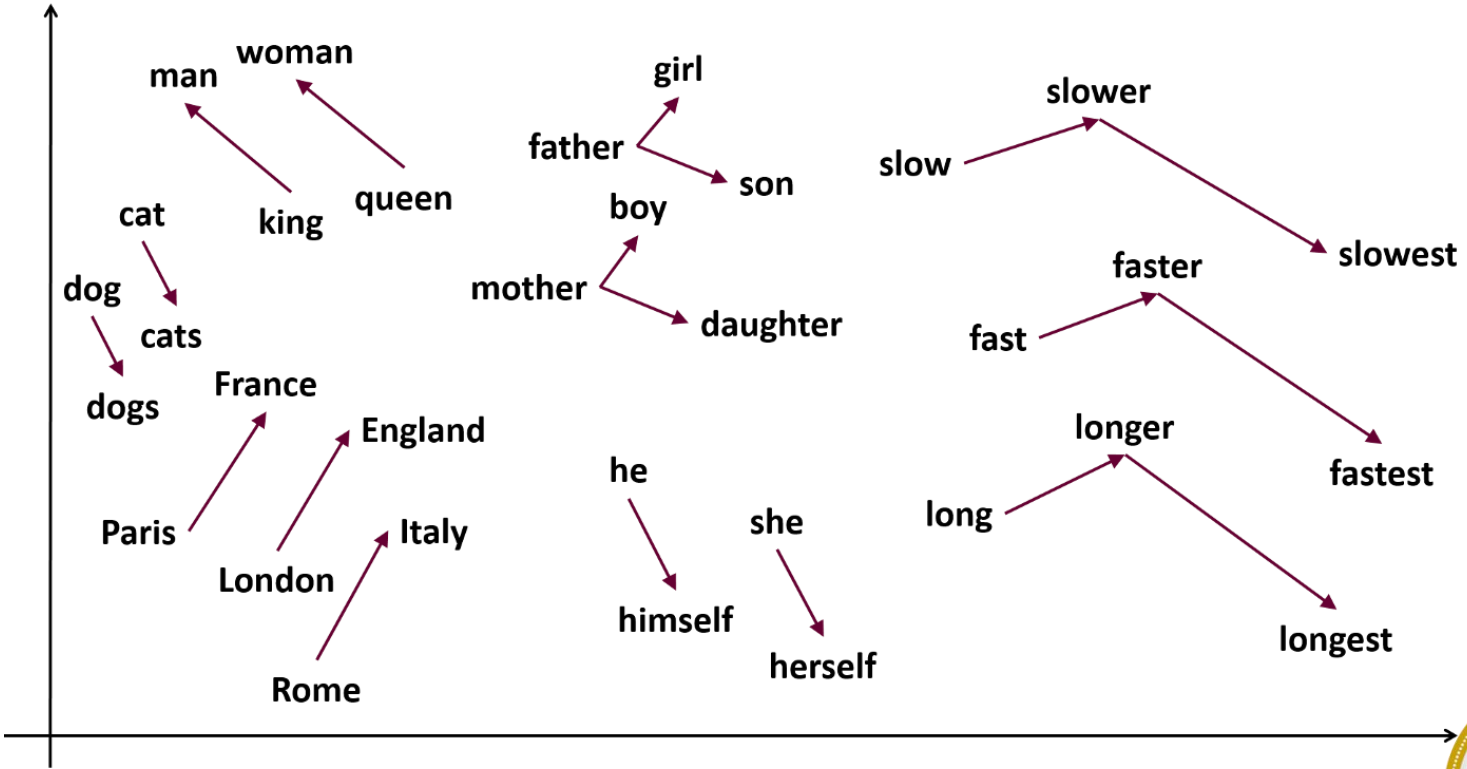
Word Embedding



Word Embedding

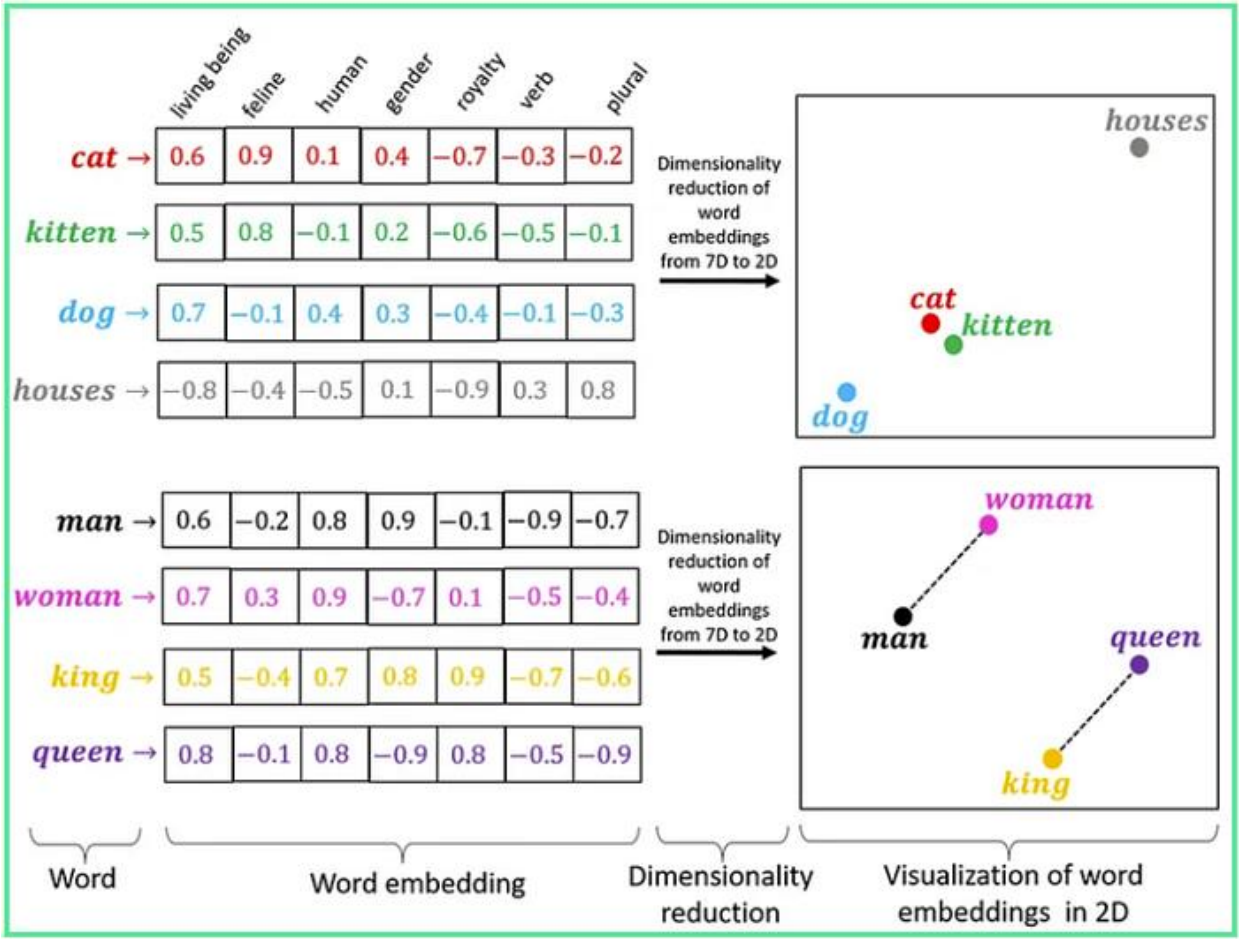


Word Embeddings

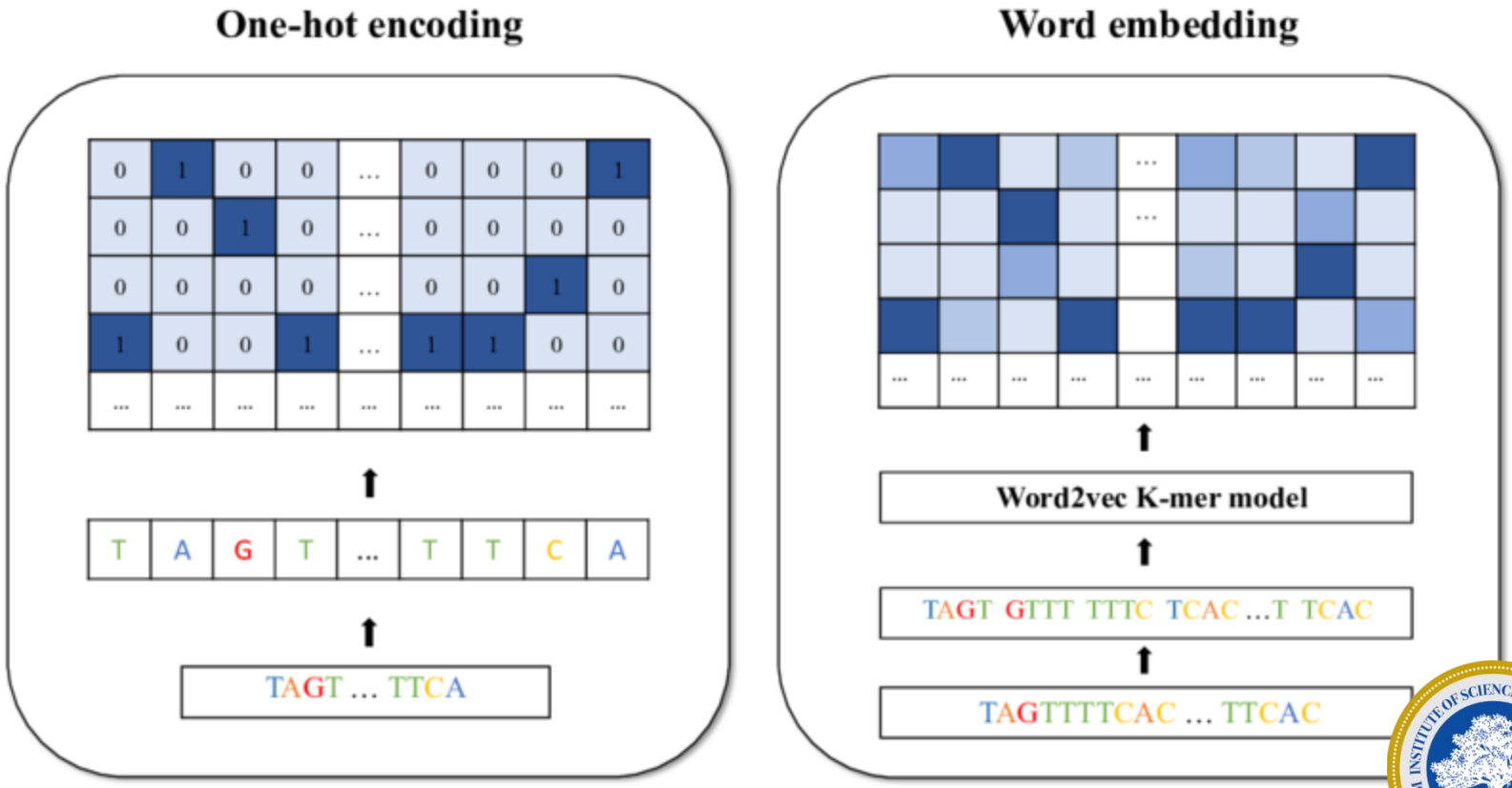


https://link.springer.com/chapter/10.1007/978-981-96-0029-8_5

Word Embeddings



Embeddings


One-hot encoding
Word embedding
T A G T ... T T C A
TAGT ... TTCA
Word2vec K-mer model
TAGT GTTT TTTC TCAC ...T TCAC
TAGTTTTCAC ... TTCAC

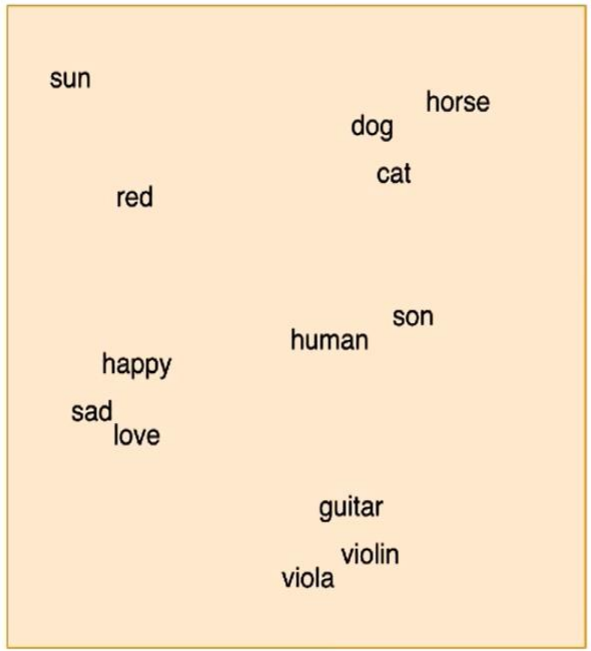
Word2Vec

- One hot encoding
- CBOW
- Skip Gram
- Many more

Word Embeddings

Input Embedding

dog →



→ $\begin{bmatrix} 0.37 \\ 0.99 \\ 0.01 \\ 0.08 \end{bmatrix}$

AJ's dog is a cutie

AJ looks like a dog

Word Embeddings

Positional Encoder :vector that gives context based on position of word in sentence

AJ's dog is a cutie → Position 2

AJ looks like a dog → Position 5

Word Embeddings

Positional Encoder :vector that gives context based on position of word in sentence

AJ's dog is a cutie → Position 2

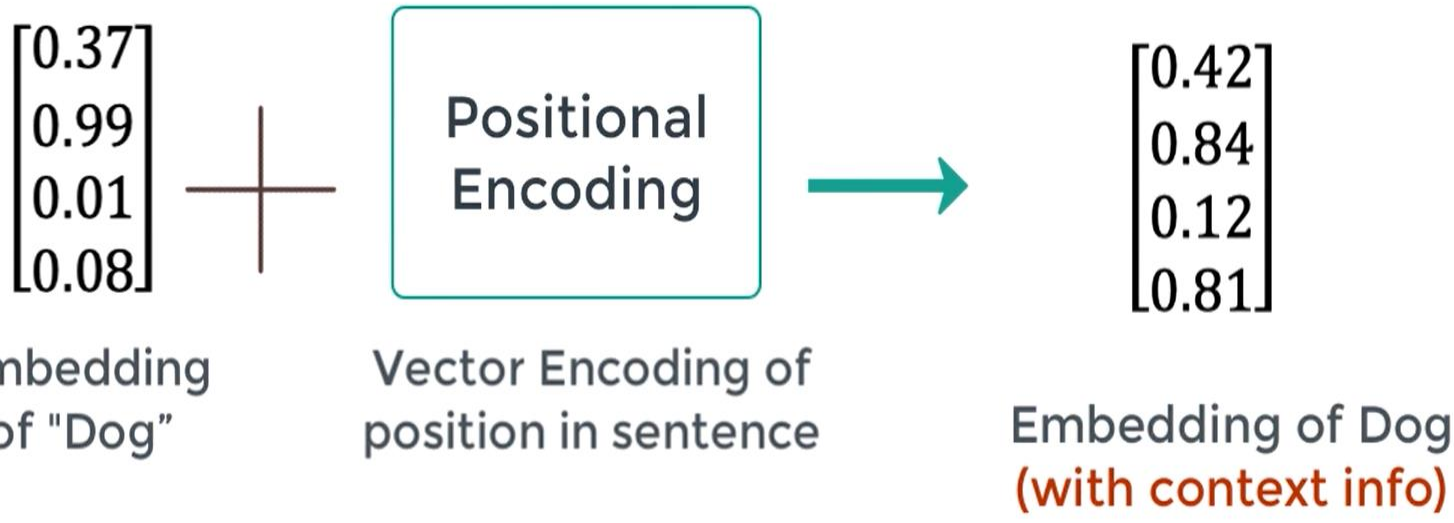
AJ looks like a dog → Position 5

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

Word Embeddings

Positional Encoder :vector that gives context based on position of word in sentence



$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

ELMo

Deep Contextualized Word Representations

***ELMo**, by Allen Institute for Artificial Intelligence, and University of Washington*

2018 NAACL, Over 8000 Citations

The "One-Word, One-Vector" Limitation

- Previous popular methods like **Word2Vec** and **GloVe** created a global dictionary of words. Each word in this dictionary was assigned a single, fixed vector representation.

- **Example:** The word "stick" would have *one* vector.
  - "Let's **stick** together." (to remain)
  - "He poked the fire with a **stick**." (a piece of wood)

- A single vector simply can't capture these different meanings. This problem is called **polysemy** (a word having multiple meanings), and it was a major limitation of static word embeddings.

ELMo: Embeddings from Language Models

- ELMo's revolutionary idea was: **Don't use a fixed dictionary. Instead, generate the embedding on the fly using the entire sentence as context.**

- By using a **Language Model (LM)**. A language model is simply a model trained to predict the next word in a sequence. By training on a massive amount of text, the LM learns deep relationships between words, syntax, and semantics.

ELMo

- ELMo uses a specific type of LM: a **deep Bidirectional Language Model (biLM)**. Let's break that down:

- **Bidirectional:** It doesn't just predict the next word (left-to-right). It also has a second model that predicts the *previous* word (right-to-left). This is crucial because a word's meaning is influenced by what comes **before** *and* **after** it.

- **Deep:** It's not just one layer. The model consists of multiple layers of LSTMs stacked on top of each other. This is important because different layers capture different types of information.

Simple Bidirectional Language Model

Given a sequence of *N* tokens, (*t*1, *t*2, …, *tN*), a **forward language model** computes the probability of the sequence by modeling the probability of token *tk* given the history (*t*1, …, *tk*-1):

$$p(t_1, t_2, \ldots, t_N) = \prod_{k=1}^{N} p(t_k \mid t_1, t_2, \ldots, t_{k-1}).$$

Simple Bidirectional Language Model

At each position k, each LSTM layer outputs a context-dependent representation:

$$\overrightarrow{\mathrm{h}}_{k,j}^{LM}$$

where j=1, …, L.
The top layer LSTM output:

$$\overrightarrow{\mathrm{h}}_{k,L}^{LM}$$

is used to predict the next token tk+1 with a Softmax layer.

Simple Bidirectional Language Model

Similarly, a **backward LM**:

$$p(t_1, t_2, \ldots, t_N) = \prod_{k=1}^{N} p(t_k \mid t_{k+1}, t_{k+2}, \ldots, t_N).$$

A **biLM** combines both a forward and backward LM. The formulation **jointly maximizes the log likelihood of the forward and backward directions**:

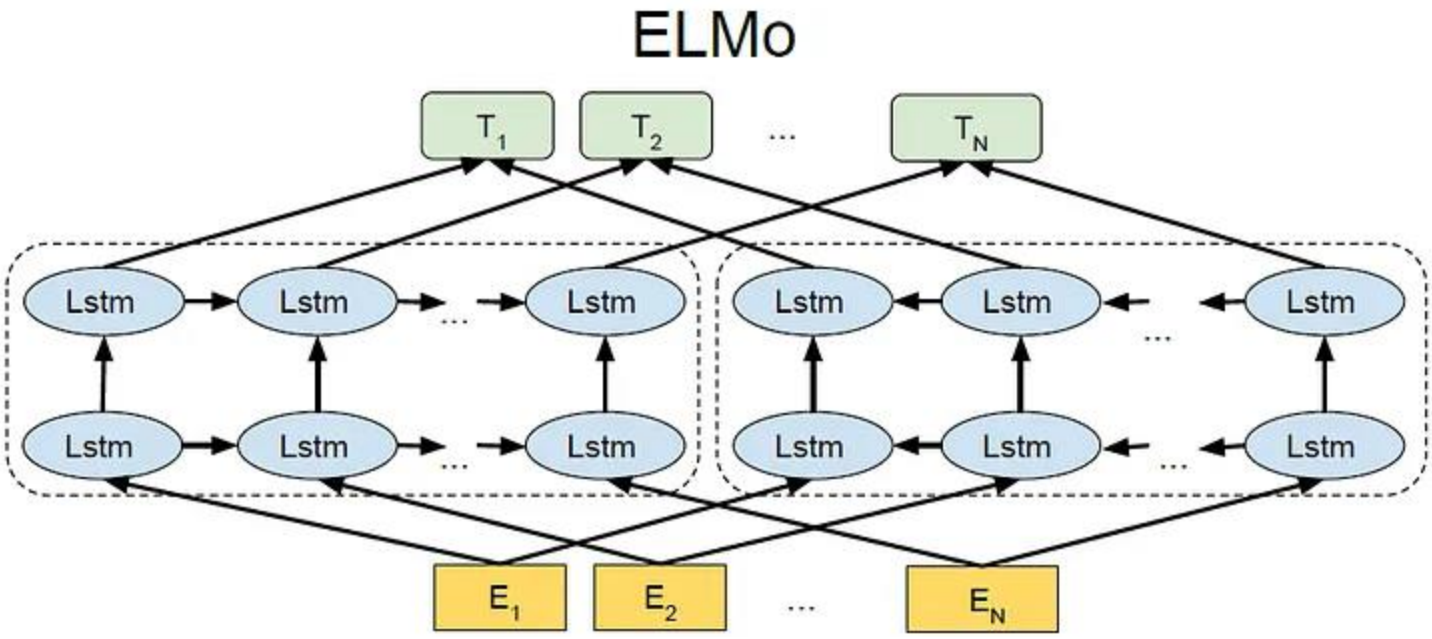
$$\sum_{k=1}^{N} \Big(\log p(t_k \mid t_1, \ldots, t_{k-1}; \Theta_x, \overrightarrow{\Theta}_{LSTM}, \Theta_s) + \log p(t_k \mid t_{k+1}, \ldots, t_N; \Theta_x, \overleftarrow{\Theta}_{LSTM}, \Theta_s) \Big).$$

Simple Bidirectional Language Model

In a bidirectional LSTM, the networks processing the forward and backward directions generally maintain their own separate sets of parameters.

However, in some configurations, specific weights may be shared between the two directions rather than keeping completely independent parameters for each.

ELMo


ELMo
T1
T2
...
TN
Lstm
Lstm
...
Lstm
Lstm
Lstm
...
Lstm
Lstm
Lstm
...
Lstm
Lstm
Lstm
...
Lstm
E1
E2
...
EN

ELMo

- **Step 1: Character-Based Input**
- Instead of looking up a word in a vocabulary, ELMo starts with the characters of the word. It uses a Character-level Convolutional Neural Network (CharCNN) to create a rough, initial embedding.
- **Benefit:** This allows ELMo to handle **out-of-vocabulary (OOV)** words. If it sees a new word like "Jedi," it can still create a reasonable vector based on its characters, rather than just marking it as "unknown."

ELMo

- **Step 2: The Deep Bidirectional LSTM**
- This is the heart of the model. The character-based embedding for each word is fed into a multi-layered biLSTM.

- **Forward LM:** One stack of LSTMs processes the sentence from left to right (e.g., "Let's go to the bank").
- **Backward LM:** A separate stack of LSTMs processes the sentence from right to left (e.g., "bank the to go Let's").

ELMo

- The final ELMo embedding for a word is not just the output of the top layer.

- Instead, it's a **weighted sum** of all the representations from all the layers for that word.

ELMo

The final ELMo vector is calculated as:

$$ELMo_k = \gamma(s_0 h_{k,0} + s_1 h_{k,1} + s_2 h_{k,2})$$

Where:

- $h_{k,0}$ is the character-based layer, and $h_{k,1}, h_{k,2}$ are the biLSTM layers.
- s_0, s_1, s_2 are **task-specific weights** that are learned during the training of the downstream model (e.g., a sentiment classifier).

- This is a brilliant feature! The downstream model learns how much syntactic vs. semantic information it needs.
  - **Lower layers** tend to capture syntax (part-of-speech, word structure).
  - **Higher layers** tend to capture semantics (context-dependent meaning).
- Gamma is a global scaling factor, also learned for the specific task.

Why ELMo Was Groundbreaking ?

- **Context is King:** It was the first widely successful model to produce truly contextual embeddings, effectively solving the polysemy problem.
- **Deep Representations:** It showed the power of using information from *all* layers of a deep network, not just the final one.
- **Character-Awareness:** Its character-based input made it robust to rare and unknown words.
- **Transfer Learning Powerhouse:** It established a powerful paradigm: pre-train a massive language model on general text, and then use its representations to significantly boost performance on specific downstream tasks (like sentiment analysis, question answering, etc.) with much less task-specific data.

Limitation of ELMo

- ELMo was powerful, but its bidirectionality was "shallow." It ran two separate LSTM models—one forward and one backward—and simply concatenated their results.

- The left-to-right model didn't know anything about the right-to-left context *while it was processing*.

- BERT's goal was to achieve **deep bidirectionality**, where the model could learn from the entire context (both left and right) simultaneously, at every single layer of the network.

Word Piece

- **Problem it solves**: Natural language has *tons* of words, plus misspellings, slang, and rare words. A simple word-based vocab would explode in size.
- **Idea**: Start with characters and iteratively merge the most frequent sequences to build *subwords*.
- **Result**:
  - Common words stay whole (play, teacher)
  - Rare/complex words get split (playing → play + ##ing, unhappiness → un + ##happy + ##ness).
- **Special thing**: Uses ## to mark subwords that are *continuations* of a word.
- **Why BERT loves it**:
  - Handles rare words (no more [UNK] everywhere).
  - Keeps vocab small (~30k tokens).
  - Works across multiple languages.

Sentence: "unhappiness"
Tokens: ["un", "##happy", "##ne

Byte-Pair Encoding (BPE) & Byte-Level BPE

- **Classic BPE**:
  - Start with characters.
  - Iteratively merge the most common *pair* of symbols into a new token.
  - Build a vocab this way.
- **Byte-level BPE** (upgrade used in GPTs):
  - Works directly at the *byte* level (0–255) instead of characters.
  - This means **any text** (Unicode, emojis, code, hashtags, foreign scripts) can be represented without special hacks.
- **Why GPT loves it**:
  - Universal (no language-specific design like WordPiece).
  - No [UNK] tokens, since *every string* is just bytes.
  - Efficient for generative models.

Sentence: "unhappiness"
Might tokenize as: ["un", "happiness"]
or depending on merges: ["un", "hap", "pi",

Word Embeddings

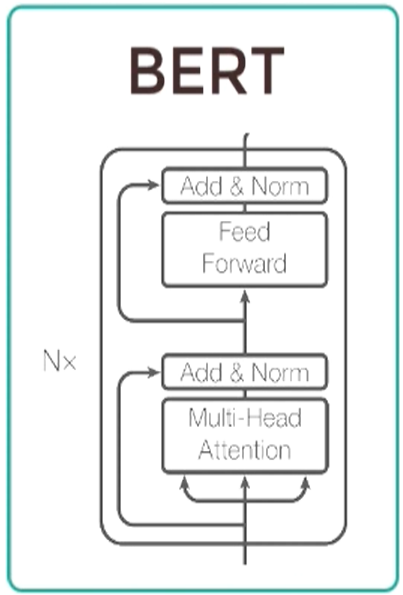
Pretraining (Pass 1) : “What is language? What is context?”

Masked Language Model (MLM)

The [MASK1] brown fox [MASK2] over the lazy dog.

Next Sentence Prediction (NSP)

A: Ajay is a cool dude.
B: He lives in Ohio



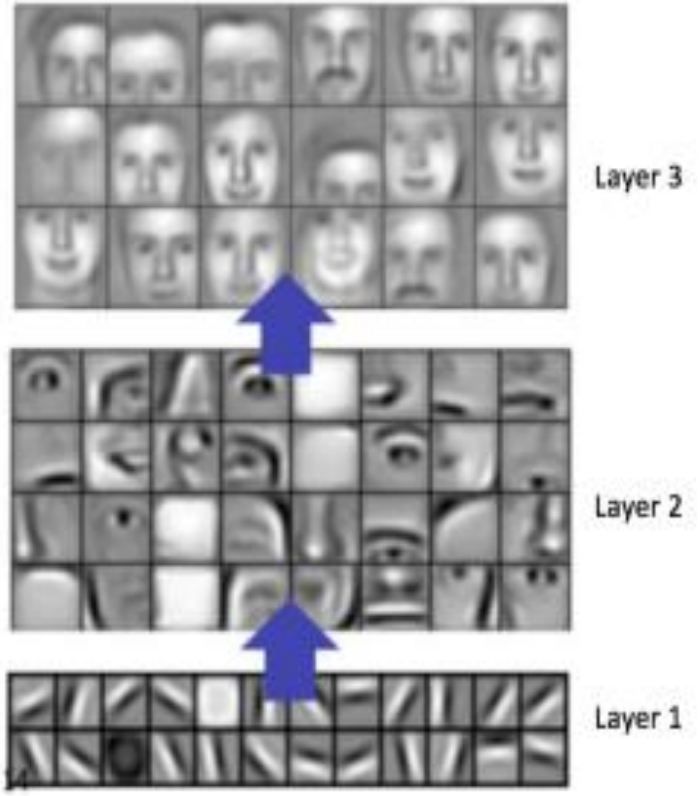
[MASK1] = quick
[MASK2]= jumped

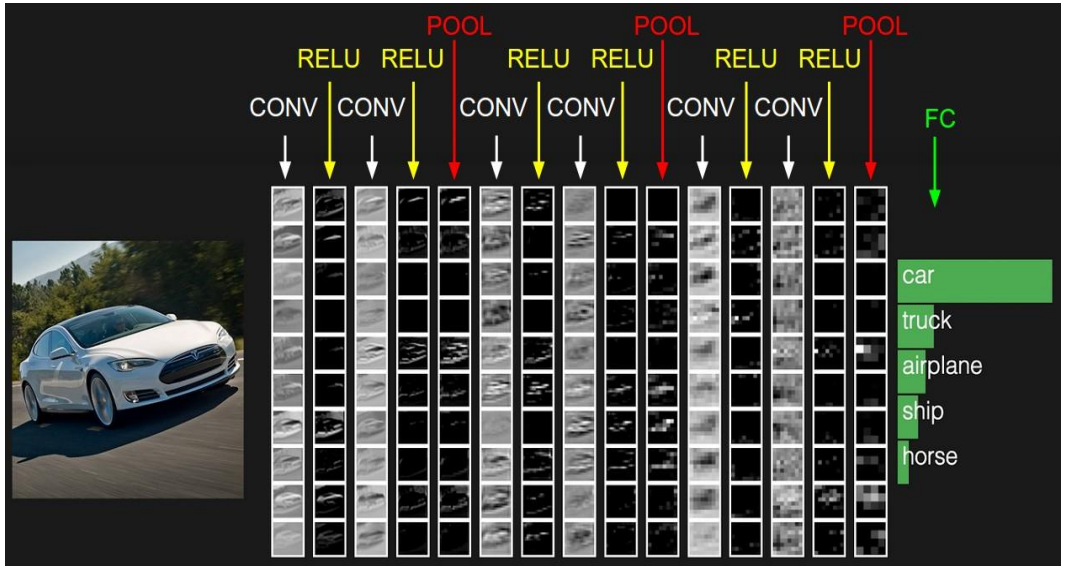
Yes. Sentence B follows sentence A

Visual Representation

CNN

Convolutional Neural Networks

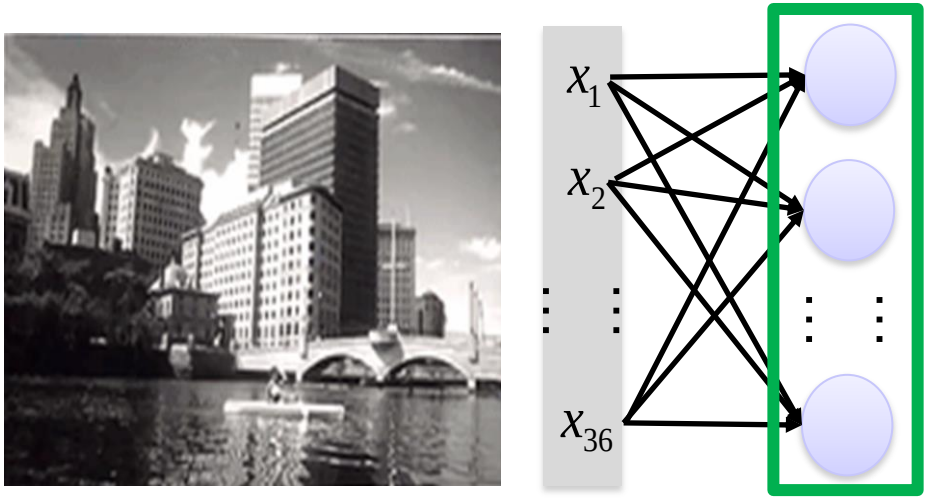




"The Detective's Magnifying Gl

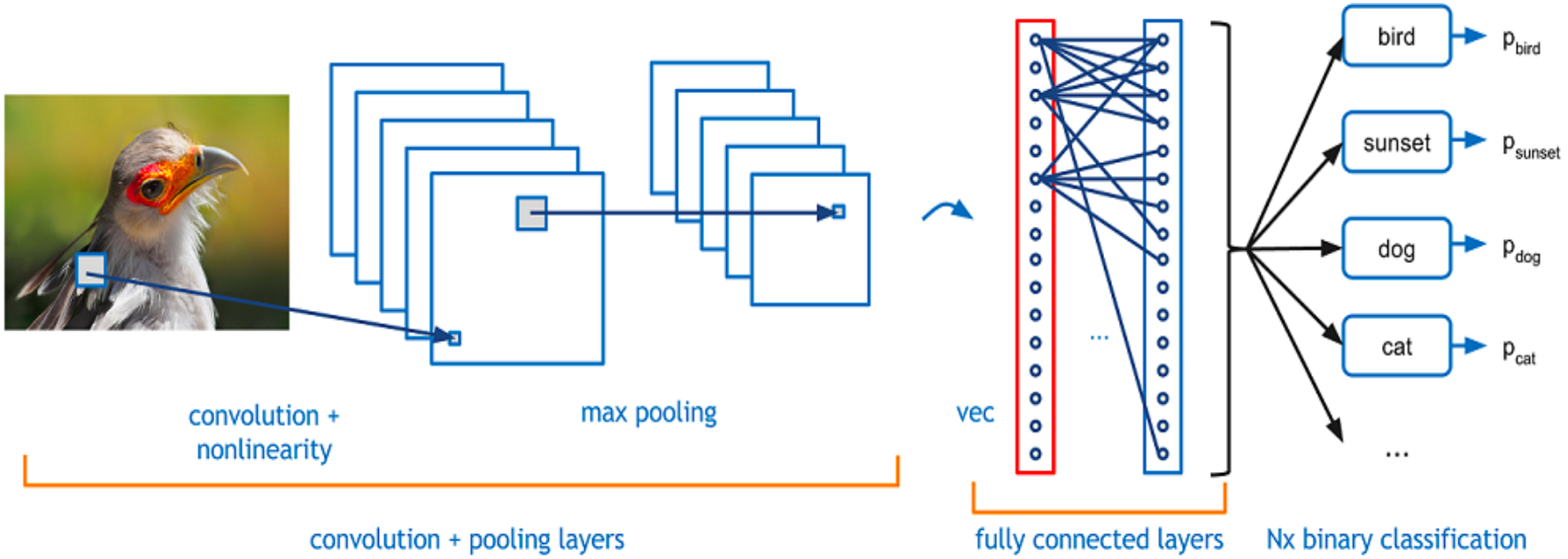
CNN VS Fully Connected Layer

Convolution

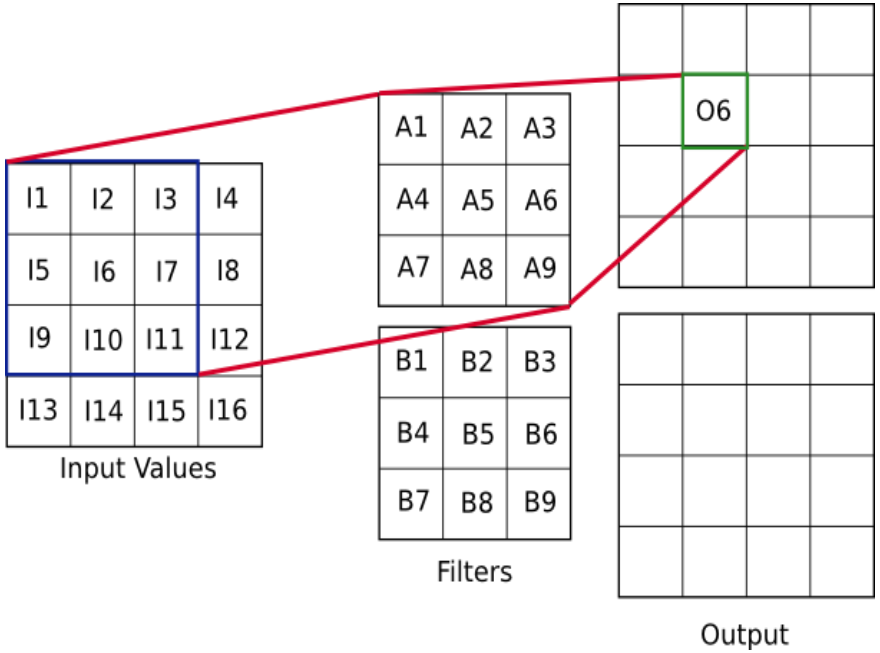


Fully-connected

Convolutional Neural Networks

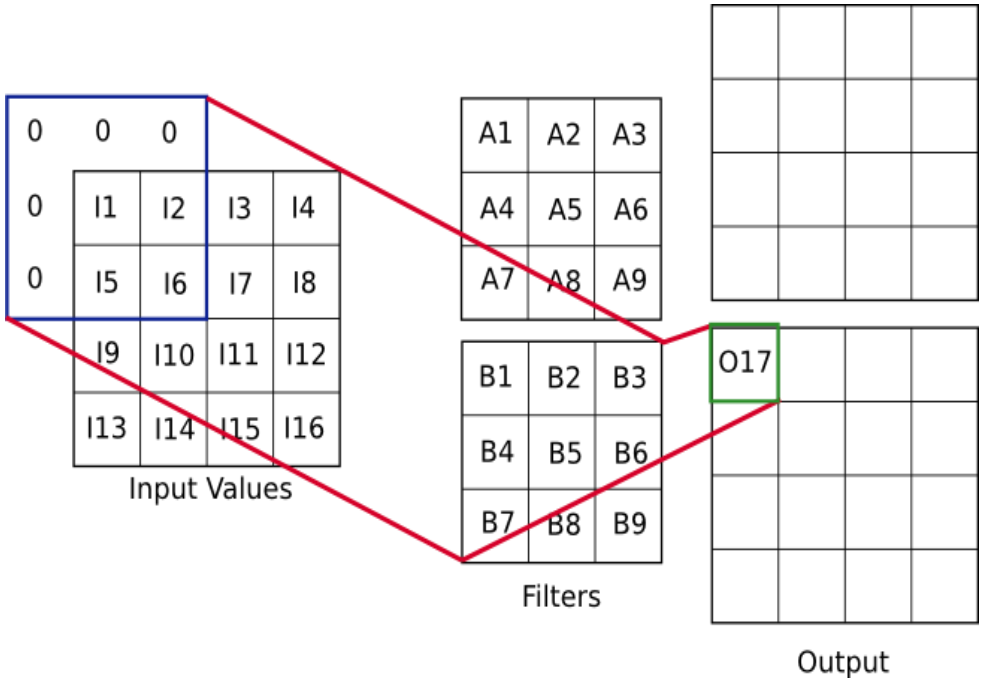


Convolutions



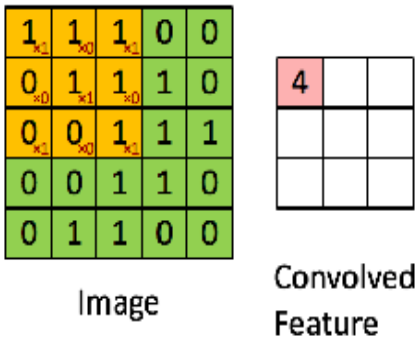
$$
\begin{aligned}
O_6 = A_1 \cdot I_1 + A_2 \cdot I_2 + A_3 \cdot I_3 \\
+A_4 \cdot I_5 + A_5 \cdot I_6 + A_6 \cdot I_7 \\
+A_7 \cdot I_9 + A_8 \cdot I_{10} + A_9 \cdot I_{11}
\end{aligned}
$$

Convolutions



$$O_{17} = B_5 \cdot I_1 + B_6 \cdot I_2 + B_8 \cdot I_5 + B_9 \cdot I_6$$

Convolution

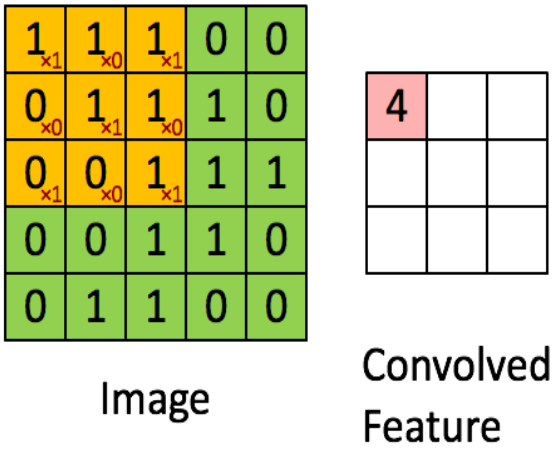


Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

[Image convolution]

Stride

- We slide the window by 1 pixel at a time. We can also slide the window by more than 1 pixel. This number is called the stride.



Convolution

Padding Layers

$$\frac{P = F - 1}{2}$$

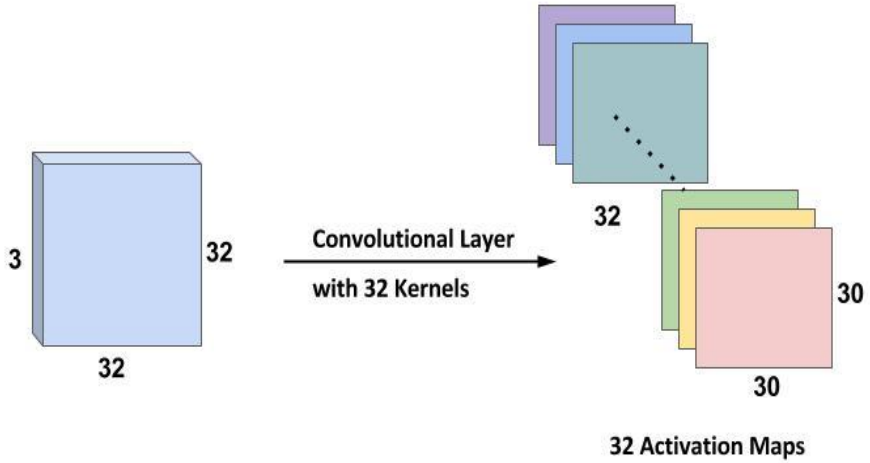
Convolution size

$$\frac{H - F + 2P}{S} + 1$$

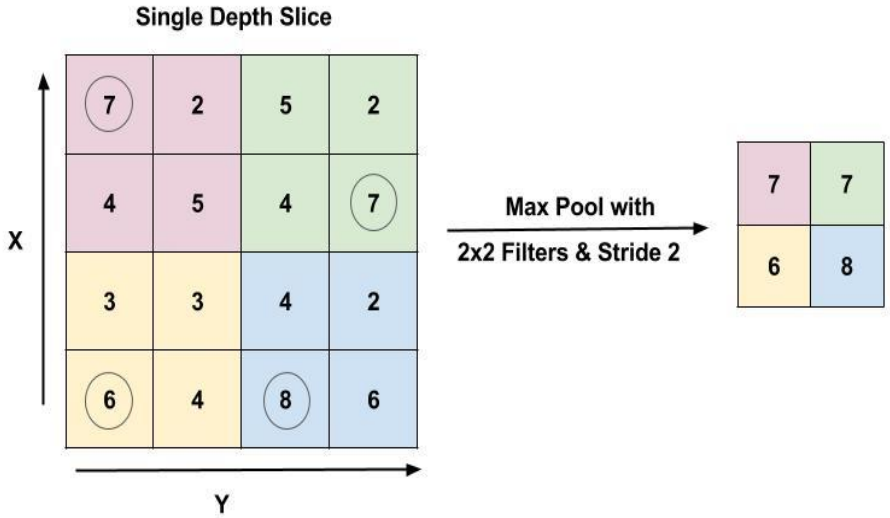
Image

0	0	0	0	0	0	0
0						0
0						0
0						0
0						0
0						0
0	0	0	0	0	0	0

- The 32 Activation maps obtained from appl wn belo



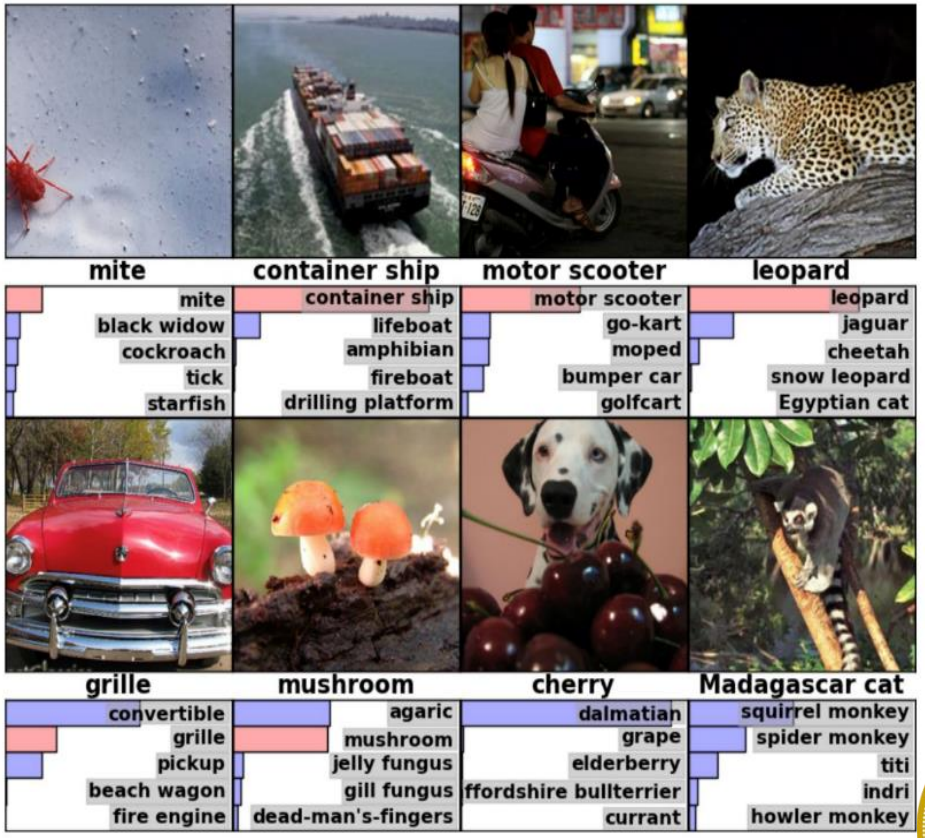
Max Pooling Layer



ImageNet Challenge

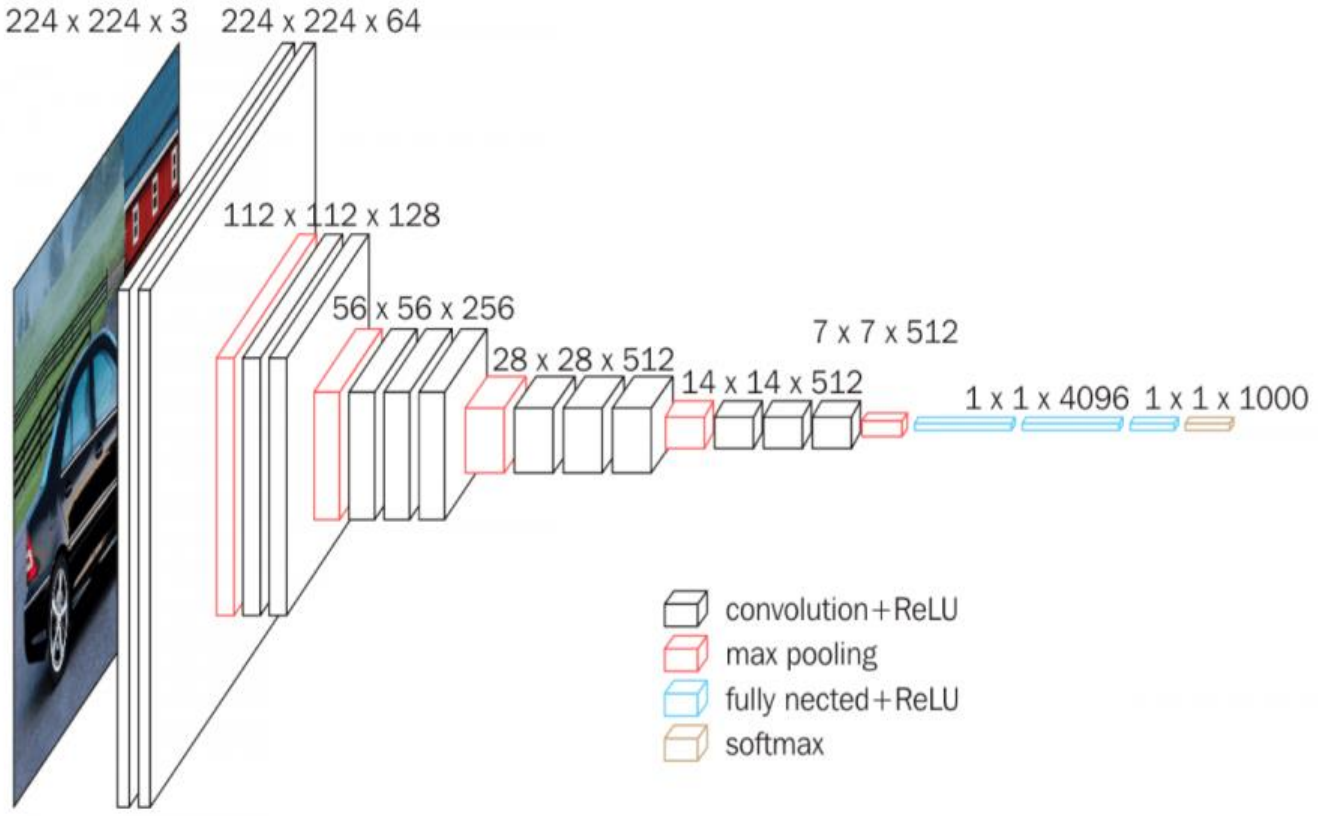
IMAGENET

- 1,000 object classes (categories).
- Images:
  - 1.2 M train
  - 100k test.



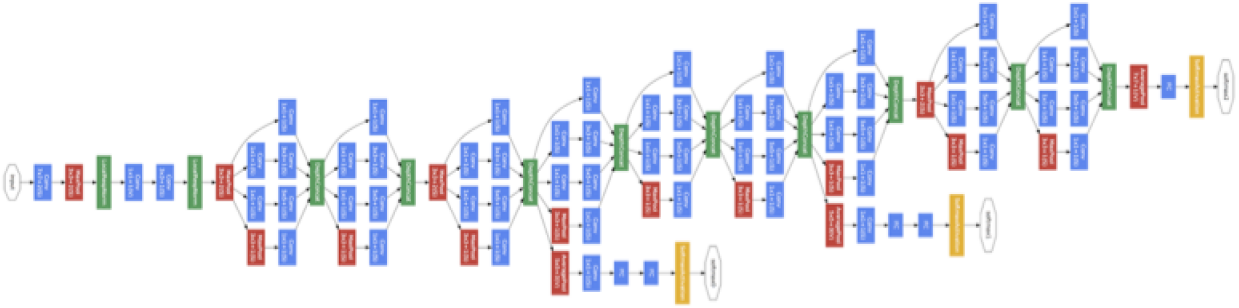
VGGNet

16 layers
All 3x3 Filters



Lesser Parameters than Alexnet
VGG 19 only Slightly better but more memory
Top Scored in Localization

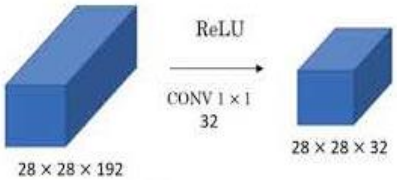
Googlenet (AKA) InceptionNet



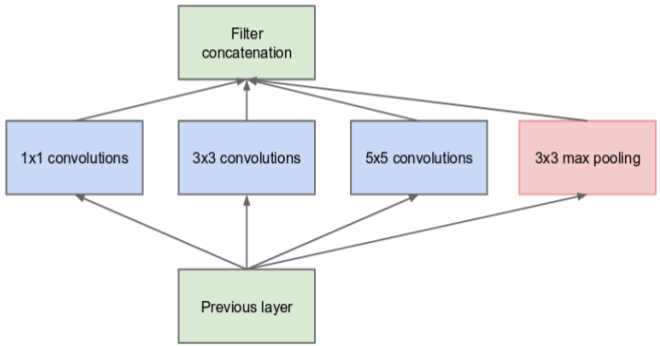
22 Layers

No FC Layers

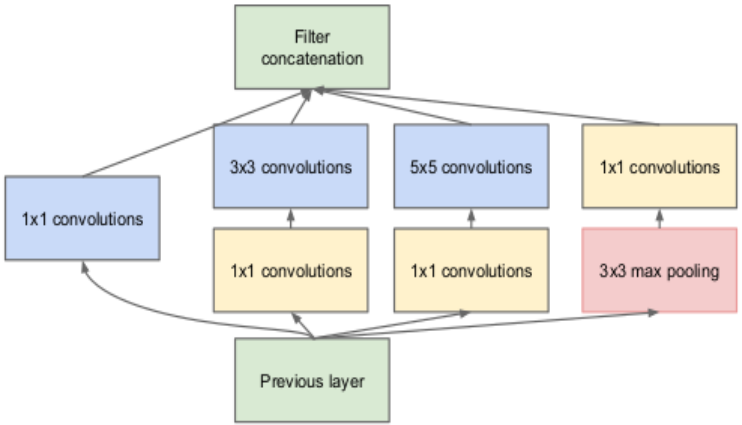
12x less than Alexnet in parameters



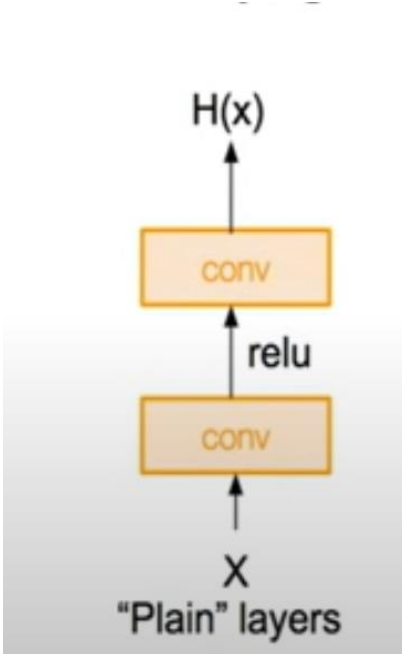
Convolution
Pooling
Softmax
Other

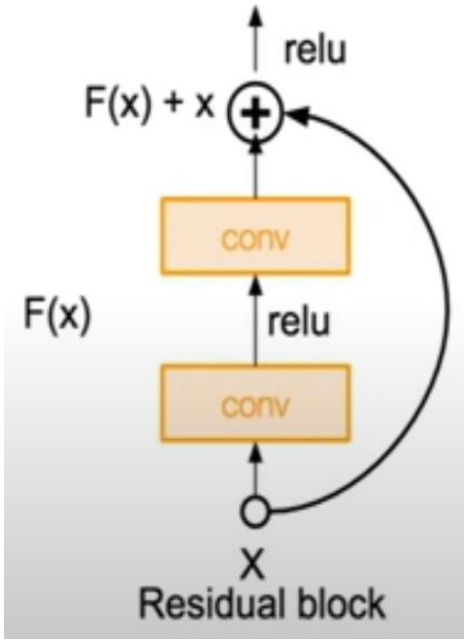


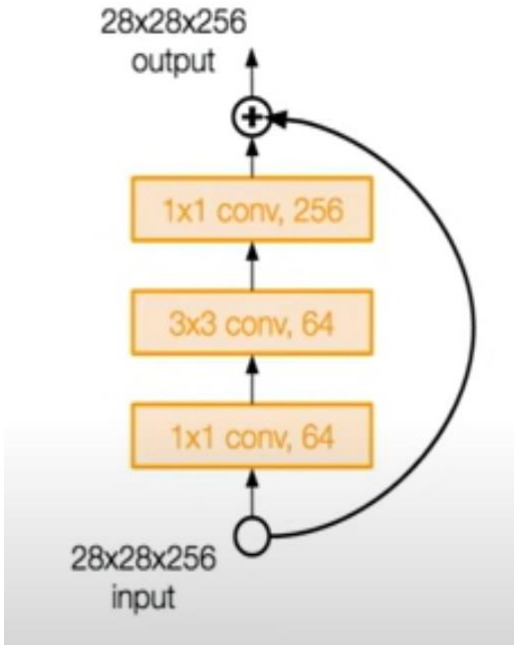
(a) Inception module, naïve version

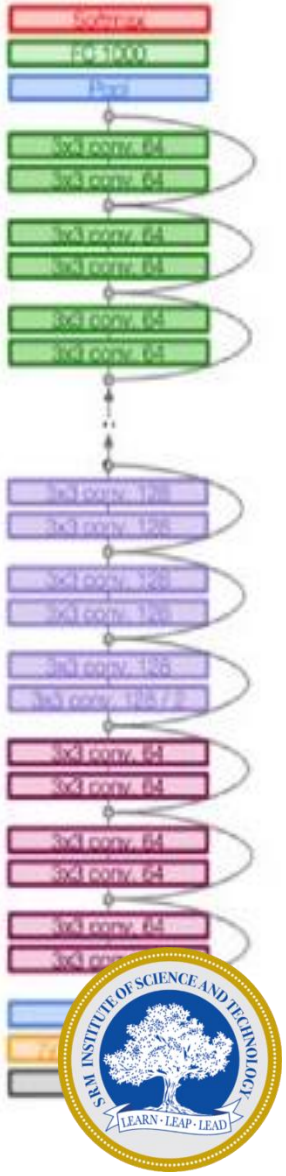


ResNet (Identity Mapping)









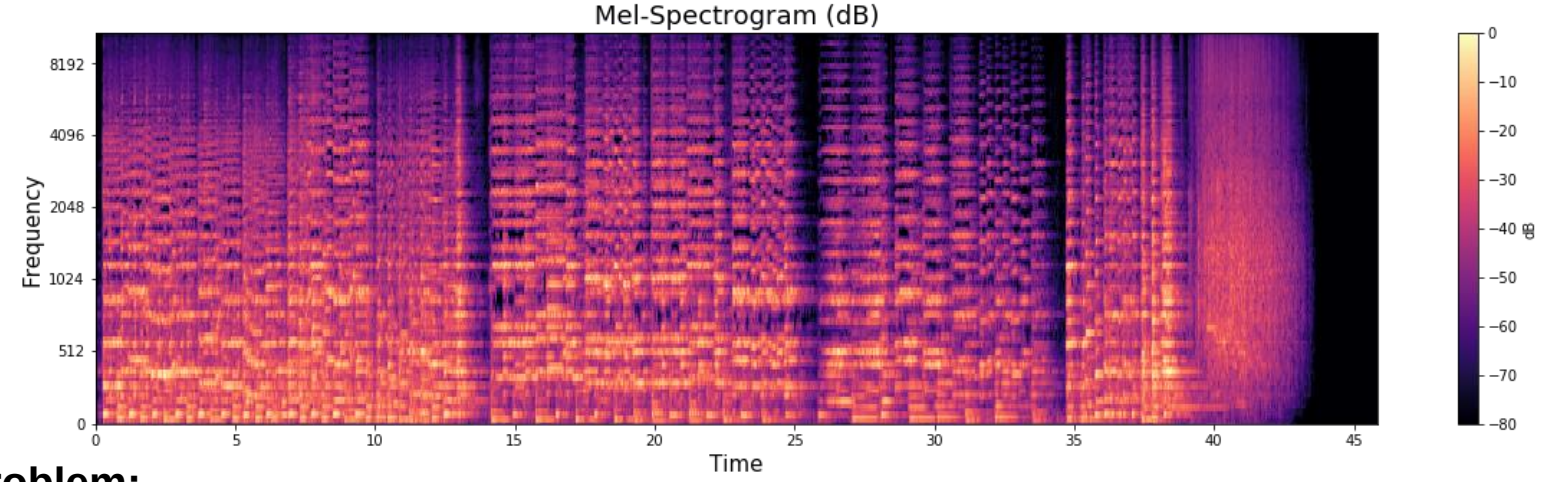
152- Layer Model

No FC and all 3x3 Filters

Acoustic Representations

SOUND PROCESSING

Sound Processing



- **The Problem:**
  - Raw audio is a **Waveform** (Amplitude vs. Time).
  - It tells us when it's loud, but not "what" the sound is.

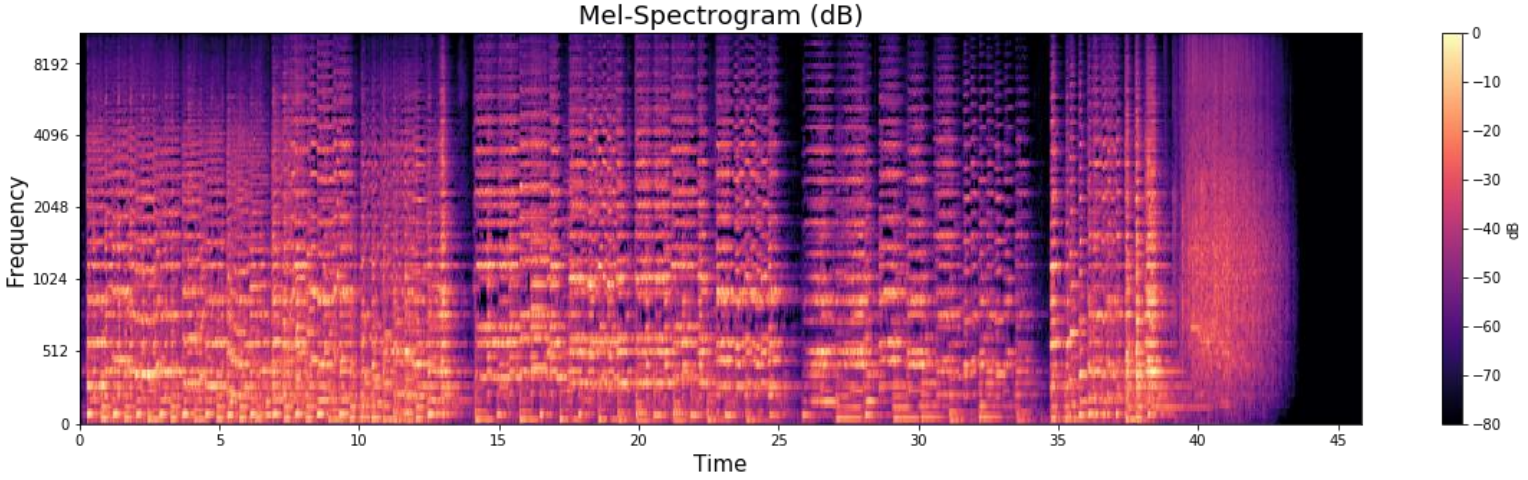
- **The Solution:** The **Spectrogram** adds a third dimension: **Frequency** (Pitch).

- **X-axis:** Time (left to right).

- **Y-axis:** Frequency (bottom is low bass, top is high treble).

- **Color (Heatmap):** Magnitude/Energy (how loud that specific frequency is at that exact ti

Transformation process

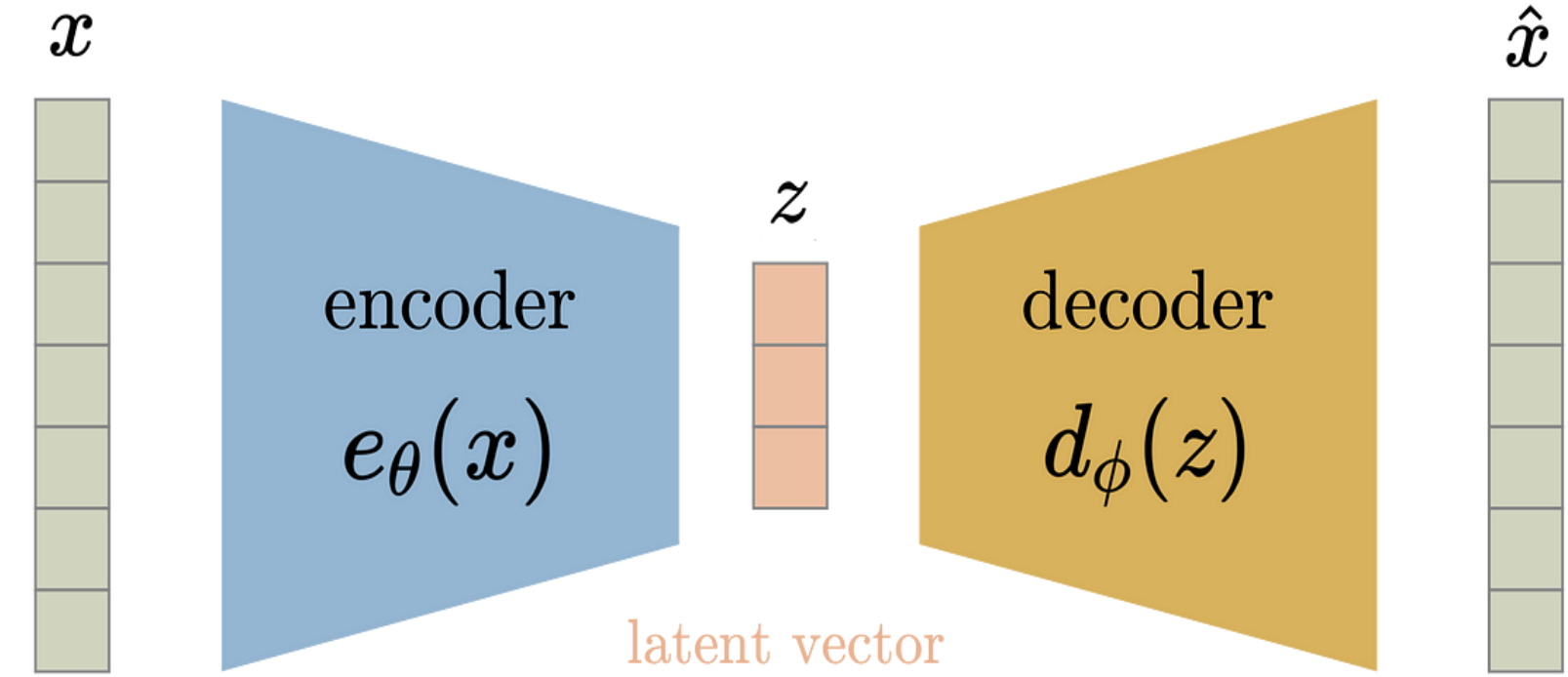


The Concept: We cannot see frequencies if we look at the whole song at once.

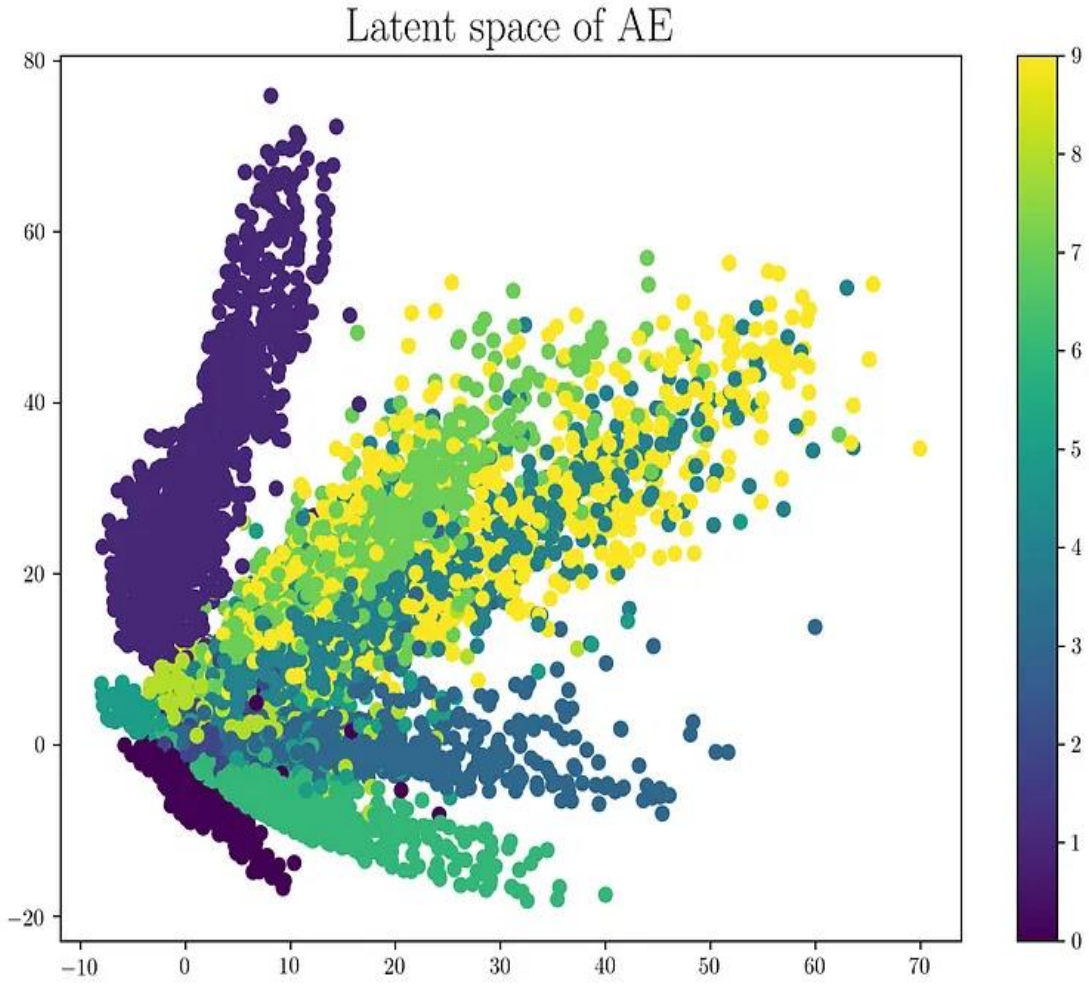
We need a "moving window".

The 3-Step Process:
Windowing: Slice the audio into tiny, overlapping chunks (e.g., 20 milliseconds).
FFT (Fast Fourier Transform): Calculate the frequency "ingredients" for that specific
Stacking: Line these slices up side-by-side to create the 2D image.

Auto Encoder

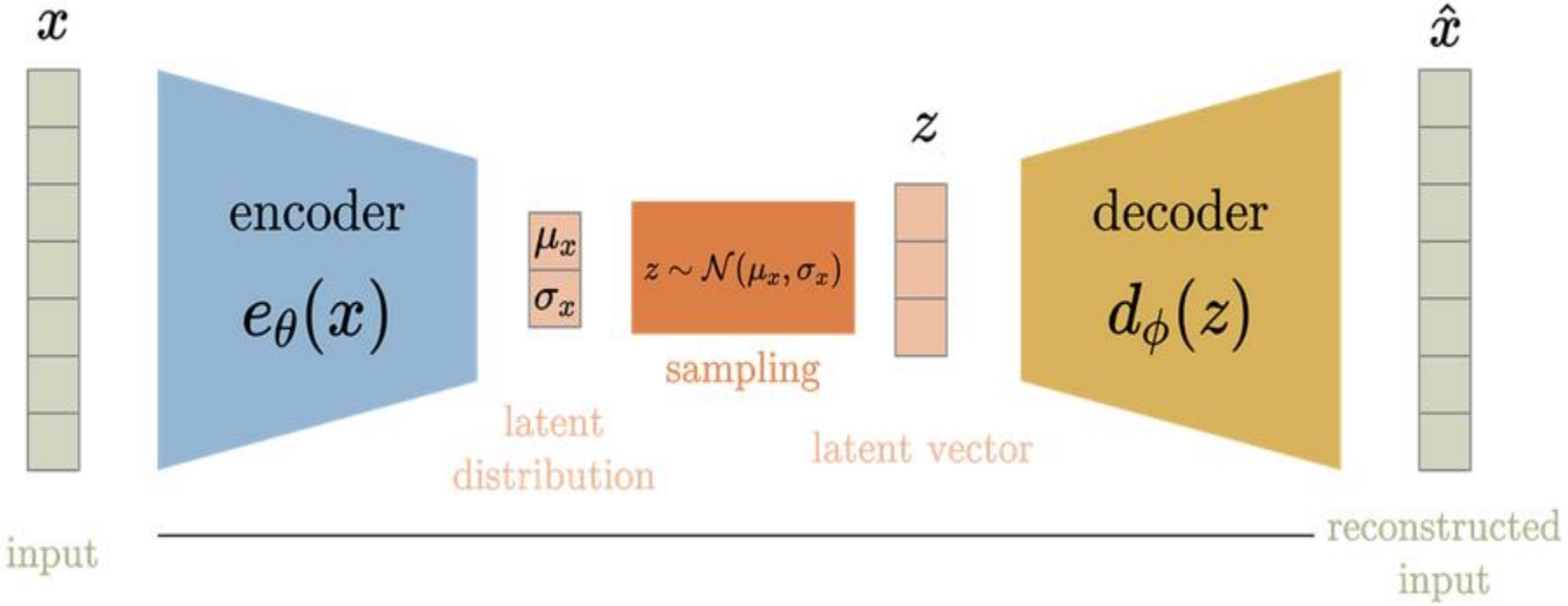


Not all the data are generative in nature



the latent space is not regularized.

Variational Auto Encoder



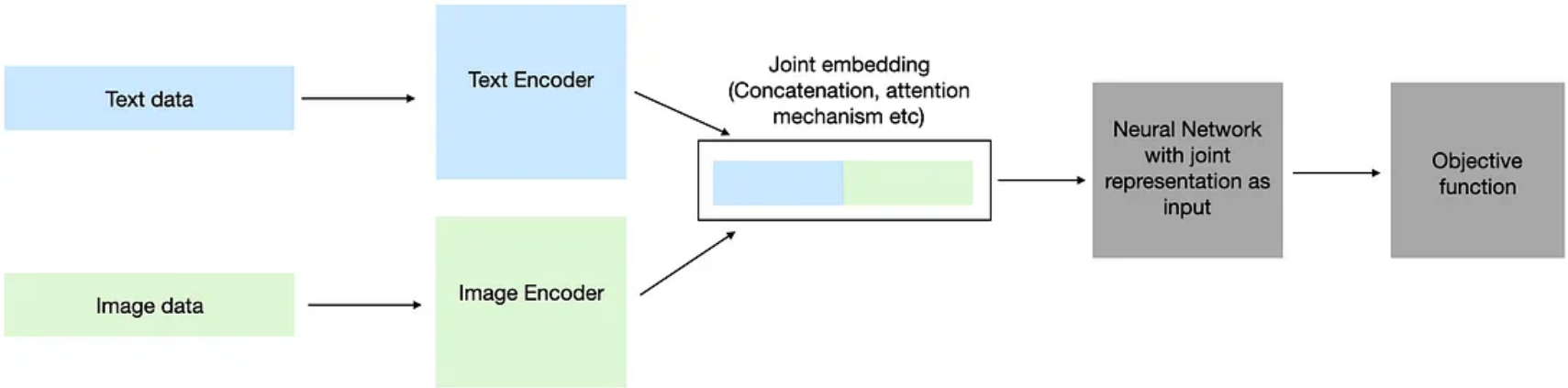
Key Difference

Feature	Standard Autoencoder (AE)	Variational Autoencoder (VAE)
Latent Mapping	Discrete, fixed points	Continuous probability "clouds"
Logic	Memorization and compression	Learning the structure and variance
Visual Result	Clusters can be irregular and scattered	Clusters are organized and centered
Main Use	Denoising, simple compression	Synthesis, generation, and robust embedding's

Multimodal Representation

- *Multi-modal learning is the ability of a machine learning model to process and learn from multiple types of data such as text, images, videos, audio, sensor data etc.*
  - *Joint Representation Learning*
  - *Co-ordinated Learning*

Join Representation Learning



Join Representation Learning

Input data: This can be combination of image, text, videos, audio, sensor depending on the problem at hand.
Encoder: The encoder is a model that converts raw data into numerical representations that the model can understand. The type of encoder used varies depending on the input data modality. See Table 1 for a list of encoders for different modalities.

Data Type	Ecoders
Text	BERT, GPT, RoBERTa, T5, Universal Sentence Encoder (USE)
Image	ResNet (Residual Networks), VGG (Visual Geometry Group), EfficientNet, Vision Transformer (ViT)
Video	I3D (Inflated 3D ConvNet):, C3D (Convolutional 3D), SlowFast Networks, R2+1D:, TSN (Temporal Segment Networks)
Audio	WaveNet, MFCC (Mel-Frequency Cepstral Coefficients), VGGish, YAMNet, SoundNet
Sensor	LSTM (Long Short-Term Memory), CNN for Time-Series Data, Temporal Convolutional Networks (TCN), Autoencoders

Join Representation Learning

Joint embedding: There are multiple ways to combine different modal embeddings.

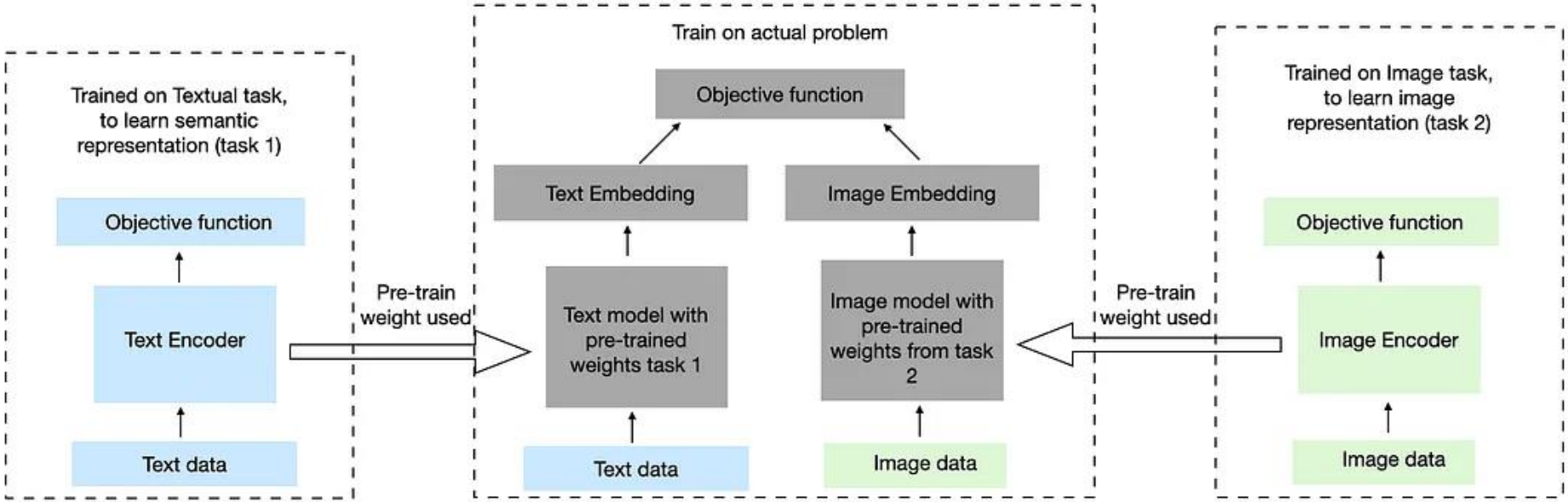
Concatenation and Projection: Concatenate the text and facial vectors, then pass through a fully connected layer (projection layer) to create a joint feature vector.

Multi-Modal Attention: Apply an attention mechanism that learns to focus on the most important features from both modalities, creating a weighted combination.

We train the model using the combined feature vector with single loss function. So Loss can be Cross-Entropy Loss (for Classification Tasks), Mean Squared Error (MSE) (for Regression Tasks), Contrastive Loss (for Embedding Learning), etc.

In summary, in joint representation learning the model is trained end-to-end, meaning that both the text and image models, along with the joint embedding layer, are trained together with a single loss function. The entire system learns to optimize the joint representation for the final task

Coordinated Learning



Coordinated Learning

- **Separate Training:** Using the same example of text and image, we initially train the text and image models independently, possibly with separate objectives or loss functions. It's essential to ensure that the separate tasks are similar to the actual task.

- **Coordinated Phase:** After separate training, during the coordination phase, the outputs from these separately trained models are combined. This combination could involve:

  - **Concatenation:** Simply combining the feature vectors from each model.
  - **Attention Mechanisms:** Learning to focus on the most important parts of each modality's output.
  - **Late Fusion:** Applying a fusion technique to integrate the outputs before making the final prediction.

When to use?

Joint Representation Learning

Joint Representation Learning combines multiple modalities into a single, unified representation. This approach is ideal for tasks where modalities are closely related, data is scarce, or computational resources are limited. By learning a shared representation, models can capture complex relationships between modalities and improve overall performance.

Coordinated Learning

Coordinated Learning trains separate models for each modality and combines their outputs for final prediction. This approach excels in tasks with distinct or separate modalities, abundant data, and sufficient computational resources. By learning separate representations, models can focus on modality-specific features and improve overall accuracy, making it suitable for tasks like multimodal sentiment analysis or language translation

Contrastive Learning

- CLIP (Contrastive Language-Image Pre-training), developed by OpenAI, is a neural network that aligns visual and textual data by training on 400 million image-text pairs.
- It uses a contrastive loss function to maximize similarity between correct pairs and minimize it for incorrect ones, allowing it to perform zero-shot tasks.
- Architecture: Consists of an image encoder (e.g., Vision Transformer) and a text encoder (e.g., Transformer) that map images and text into a shared 512-dimensional embedding space.
- Contrastive Learning: During training, the model receives a batch of (N) image-text pairs and learns to align them, considering the (N) pairs as positive examples and the (N^{2}-N) incorrect combinations as negative examples.
- Loss Function: A symmetric cross-entropy loss is applied, calculating the loss for both the image-to-text direction and text-to-image direction, then averaging them.
- Zero-Shot Capabilities: Because it understands the connection between text and visual concepts, CLIP can classify images based on new, arbitrary text descriptions without specific training.

Visual Semantic Spaces

- **Visual Semantic Spaces** are a concept in multimodal machine learning where **images** (visual data) and **text** (semantic data) are mapped into a single, shared mathematical space (a vector space).
- In this shared space, an image and its corresponding text label are forced to be "neighbors.“

- In traditional computer vision, a model classifies an image as Class 34 (which happens to be "Dog"). It doesn't know what a "Dog" is; it just knows it is distinct from Class 35.

- In a **Visual Semantic Space**, the model learns that the **image of a dog** is mathematically similar to the **word vector for "dog"**.

Multimodal Autoencoder

- A Multimodal Autoencoder (MAE) is a type of neural network designed to learn a compressed, joint representation of data that comes from multiple sources (modalities), such as Image + Text or Audio + Video.

- Unlike a standard autoencoder that reconstructs one input (e.g., Image → Compressed Code →Image), a multimodal autoencoder takes multiple inputs, fuses them into a single shared "code" (latent representation), and then tries to reconstruct the original inputs from that shared code.

Multimodal Autoencoder

- A Multimodal Autoencoder typically consists of three parts:

- **Encoders (Feature Extraction):** You have separate neural networks for each modality.
  - **Image Encoder:** A CNN (like ResNet) takes an image and outputs a vector.
  - **Text Encoder:** A model like LSTM or Transformer takes text and outputs a vector.

- **Shared Representation Layer (Fusion):** The outputs from the individual encoders are concatenated (joined together) and passed through a few dense layers to create a single, compressed "Joint Code" . This code contains the essence of *both* the image and the text.

- **Decoders (Reconstruction):** The network then splits back into two separate paths to reconstruct the original inputs.
  - **Image Decoder:** Takes the joint code and tries to recreate the original image.
  - **Text Decoder:** Takes the joint code and tries to recreate the original text.

Multimodal Autoencoder

- The true power of a Multimodal Autoencoder is its ability to handle **missing data**.
- If you train the model correctly (often using techniques like "Modality Dropout"), the shared representation becomes robust.

- **Scenario:** You only have the **Image** (e.g., a photo of a dog), but the **Text** is missing.

- **Process:** The model encodes the image into the joint space.

- **Result:** The **Text Decoder** can still generate the missing text ("A photo of a dog") because the joint code has learned the relationship between visual features and words.

- This is fundamentally how early **Image Captioning** and **Cross-Modal Retrieval** systems were built.

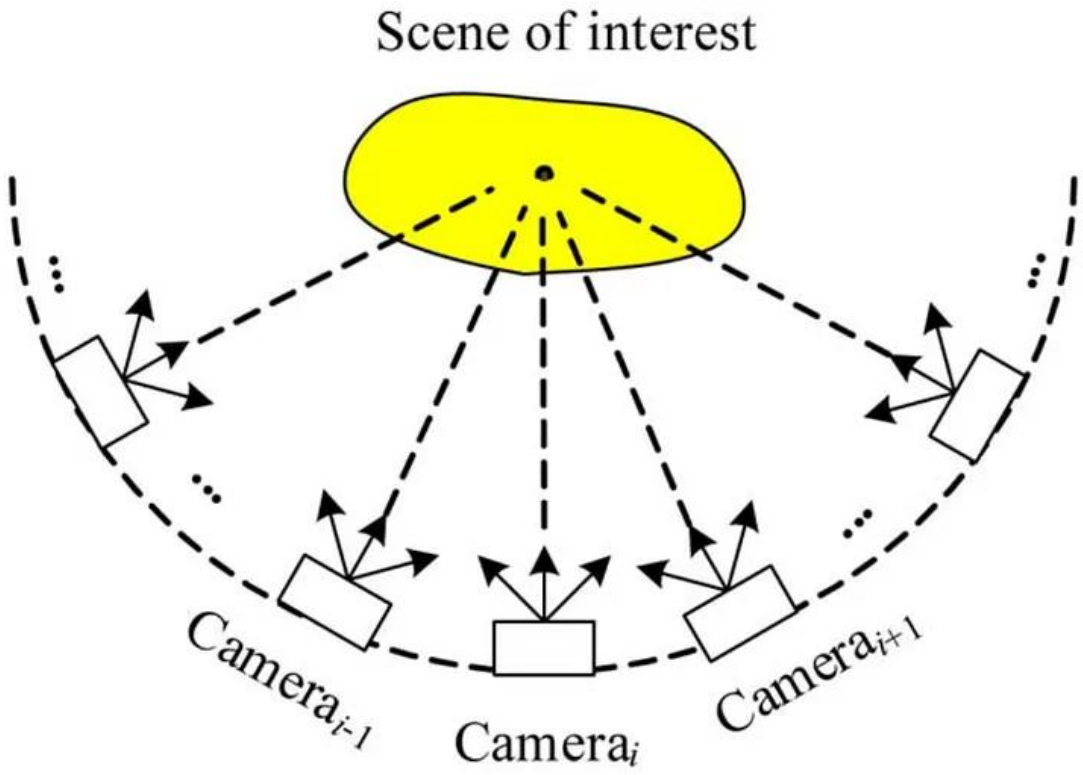
Orthogonal Joint Representation

- **Orthogonal Joint Representation** is an advanced method in multimodal learning where the goal is not just to "fuse" data, but to **disentangle** it.

- It mathematically forces the model to separate information into two distinct buckets:
  - **Shared Features:** Information present in *both* modalities (e.g., the person's identity in a video and their voice).
  - **Private Features:** Information unique to *only one* modality (e.g., the lighting in the video, or the background noise in the audio).

- The "Orthogonal" part comes from the mathematical constraint: we force the **Shared Vector** and the **Private Vector** to be perpendicular (statistically uncorrelated), ensuring no information leaks between them.

Canonical Correlation Analysis



End of Slide

THANK YOU